PERTEMUAN 13 · Praktik terbimbing · Kuis konsep

Analisis Semantik: Attribute Grammar, SDD, dan Translation Scheme

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 01 — Menjelaskan tahapan kompilasi (*lexical*, *syntax*, *semantic analysis*).

Sub-CPMK yang dinilai pada pertemuan ini

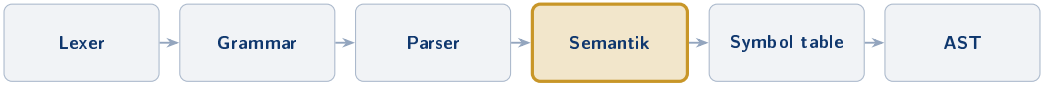
Sub-CPMK 1.5: Membuat tabel aturan semantik berbasis *syntax-directed definition* untuk ekspresi dan pernyataan subset bahasa C, lalu menguji keluarannya pada pohon sintaks.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
25	Ceramah: ruang lingkup semantik dan attribute grammar
30	Latihan: atribut *synthesized* dan *inherited*
35	Lokakarya: tabel SDD ekspresi dan pernyataan
25	Latihan: menguji aturan pada pohon sintaks
20	Kuis konsep dan pembahasan
15	Kode antara: postfix dan three-address, lalu penutup

Porsi latihan dan praktik 90 menit; kuis konsep 20 menit. Keluaran berupa tabel aturan dan pohon teranotasi.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini membangun simpul **semantik**: memeriksa makna program di atas AST.
- Tabel aturan semantik menjadi dasar implementasi *type checker* pada Pertemuan 14.
- Keluaran berupa tipe dan error yang dipropagasikan.

Tugas Analisis Semantik

Tugas	Pertanyaan yang dijawab
Resolusi nama	Apakah identifier sudah dideklarasikan?
Pemeriksaan tipe	Apakah operand memiliki tipe yang kompatibel?
Pemeriksaan fungsi	Apakah jumlah dan tipe argumen sesuai?
Kontrol alur	Apakah `break` berada di dalam loop?
Inisialisasi	Apakah variabel dipakai sebelum diberi nilai?
Keterjangkauan	Apakah semua jalur fungsi mengembalikan nilai?

Semua tugas ini berada di luar daya ekspresif CFG; diperlukan tabel simbol dan aturan tipe.

Informasi dan Urutan Analisis

Informasi yang dibutuhkan

- AST atau pohon urai;
- tabel simbol dengan scope dan tipe;
- aturan tipe setiap operator;
- mekanisme pelaporan kesalahan.

Urutan analisis

1. bangun tabel simbol dan resolusi nama;
2. tentukan tipe setiap ekspresi;
3. periksa kecocokan tipe;
4. periksa kontrol alur;
5. laporkan kesalahan dan lanjutkan.

Analisis tidak menghasilkan kode; tujuannya memastikan program valid dan mengumpulkan informasi untuk fase berikutnya.

Attribute Grammar

Definisi

Attribute grammar adalah tata bahasa bebas konteks yang setiap simbolnya memiliki atribut dan setiap produksinya memiliki aturan semantik untuk menghitung atribut tersebut.

- Atribut dapat berupa tipe, nilai, alamat, kode, atau informasi lain.
- Setiap atribut memiliki domain nilai dan aturan evaluasi.
- Ketergantungan antar-atribut membentuk graf berarah.
- Evaluasi mengikuti urutan topologis graf; siklus harus dihindari.

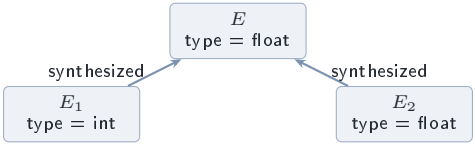
Atribut *Synthesized* dan *Inherited*

Synthesized

Dihitung dari atribut anak. Nilai mengalir dari daun ke akar.
Contoh: $E.type = \text{rule}(E_1.type, E_2.type)$.

Inherited

Dihitung dari orang tua atau saudara. Nilai mengalir dari akar ke daun atau antar-saudara.
Contoh: $\text{stmt}.env = \text{block}.env$.



Syntax-Directed Definition (SDD)

Definisi

SDD adalah tata bahasa dengan atribut dan aturan semantik. Setiap produksi $A \to \alpha$ memiliki aturan untuk menghitung atribut simbol-simbolnya.

- SDD mendeskripsikan *apa* yang dihitung, bukan *kapan* perhitungan dilakukan.
- Urutan evaluasi ditentukan oleh ketergantungan antar-atribut.
- SDD S-attributed hanya memakai atribut synthesized; dapat dievaluasi post-order.
- SDD L-attributed memakai inherited yang bergantung pada kiri atau orang tua; dievaluasi kiri-ke-kanan.

Tabel SDD untuk Ekspresi (Artefak Sub-CPMK 1.5)

Produksi	Aturan semantik
$E \to E_1 + E_2$	$E.type = \text{arith}(E_1.type, E_2.type)$
$E \to E_1 * E_2$	$E.type = \text{arith}(E_1.type, E_2.type)$
$E \to \texttt{id}$	$E.type = \text{lookup}(\texttt{id}.name)$
$E \to \texttt{INT_LIT}$	$E.type = \text{int}$
$E \to \texttt{FLOAT_LIT}$	$E.type = \text{float}$
$E \to (E_1)$	$E.type = E_1.type$

`arith` mengembalikan `float` jika salah satu operand float, `int` jika keduanya int/char, dan `error` jika operand tidak numerik.

Tabel SDD untuk Pernyataan

Produksi	Aturan semantik	Keluaran uji
$S \to \texttt{id} = E$	$S.type = \text{check_assign}(\text{lookup}(\texttt{id}), E.type)$	tipe S atau error
$S \to \texttt{if}(E)\ S_1$	$\text{check_bool}(E.type);\ S.type = S_1.type$	tipe S
$S \to \texttt{while}(E)\ S_1$	$\text{check_bool}(E.type);\ S.type = S_1.type$	tipe S
$S \to \{S_1\ S_2\}$	$S.type = \text{seq}(S_1.type, S_2.type)$	tipe S
D deklarasi	$\text{declare}(\texttt{id}, \texttt{type})$	entri tabel simbol

Tabel ini melengkapi tabel ekspresi; keduanya menjadi artefak Sub-CPMK 1.5.

Menguji Aturan pada Pohon Sintaks

```
int a = 2;
float b = 3.5;
a = b + 1;
```

Listing 1: Program uji aturan semantik

Simpul	Aturan	Hasil
Deklarasi a	declare(a, `int`)	a : `int`
Deklarasi b	declare(b, `float`)	b : `float`
$b + 1$	arith(float, int)	float
$a = b + 1$	check_assign(int, float)	error

Setiap baris merujuk simpul pohon konkret, bukan hanya nama aturan.

S-Attributed dan L-Attributed

Aspek	S-attributed	L-attributed
Atribut	Synthesized	Synthesized dan inherited
Evaluasi	Post-order	Kiri-ke-kanan
Implementasi	Mudah	Perlu konteks
Contoh	Tipe ekspresi	Lingkungan scope

Evaluasi pada proyek Subset-C

Evaluasi dilakukan setelah AST terbentuk, dengan traversal. Pendekatan ini memisahkan parsing dari analisis dan memudahkan pengujian.

Translation Scheme

Definisi

Translation scheme adalah tata bahasa dengan aksi semantik yang ditempatkan pada posisi tertentu dalam produksi. Aksi dijalankan ketika posisi tersebut dicapai selama parsing.

Contoh

$E \to E_1 + \{\ E.type = \text{arith}(E_1.type, E_2.type);\ \} E_2$.
Aksi dijalankan setelah E_1 dan sebelum E_2 diproses.

SDD mendeskripsikan aturan; translation scheme mendeskripsikan urutan eksekusi.

Aksi Semantik pada Bison

```
expr : expr T_PLUS expr
     {
         $$ = ast_new(AST_BINARY, $2.token);
         $$->left = $1;
         $$->right = $3;
     }
     ;
```

Listing 2: Translation scheme pada Bison

- Aksi dijalankan saat aturan direduksi.
- Kedua operand sudah tersedia.
- Pola S-attributed yang khas.
- Logika kompleks dipindahkan ke fungsi bantu.
- Grammar tetap ringkas dan dapat diuji.

Modul Tipe `types.h`

```
7  typedef enum {
8      TYPE_INT,
9      TYPE_FLOAT,
10     TYPE_CHAR,
11     TYPE_VOID,
12     TYPE_STRING,
13     TYPE_BOOL,
14     TYPE_ERROR
15 } TypeKind;
16
17 typedef struct {
18     TypeKind kind;
19 } Type;
20
21 const char *type_name(TypeKind kind);
```

Listing 3: Enum dan struktur tipe pada `types.h`

- `TYPE_ERROR` sebagai nilai propagasi.
- Jika satu operand error, hasil juga error.
- Kesalahan tidak berlipat ganda.
- `type_arith` dan `type_assignable` mengimplementasikan aturan.

Notasi Postfix

- Operator ditulis setelah operand.
- $(a + b) * (c + d)$ menjadi `a b + c d + *`.
- Evaluasi memakai stack: operand didorong; operator mengambil dua operand teratas, lalu mendorong hasilnya.

Infix	Postfix	Catatan
`a + b`	`a b +`	satu operator
`a + b * c`	`a b c * +`	perkalian dahulu
`(a + b) * c`	`a b + c *`	kurung mengubah urutan
`a = b + c * d`	`b c d * + a =`	assignment

Postfix tidak memerlukan tanda kurung karena urutan operasi ditentukan posisi operator.

N-Tuple dan Three-Address Code

Quadruple untuk `a = b + c * d`

No	Op	O1	O2	Hasil
1	*	c	d	t1
2	+	b	t1	t2
3	=	t2	—	a

Three-address code

```
t1 = c * d
t2 = b + t1
a = t2
```

- Triple merujuk hasil antara dengan nomor; quadruple menyimpan nama hasil.
- Three-address code dekat dengan instruksi assembly.

Kuis Konsep

1. Apa perbedaan atribut *synthesized* dan *inherited*?
2. Mengapa CFG tidak dapat memeriksa apakah identifier sudah dideklarasikan?
3. Tulis aturan SDD untuk tipe $E \to E_1 + E_2$.
4. Apa fungsi `TYPE_ERROR` dalam propagasi error?
5. Kapan *translation scheme* lebih praktis daripada SDD?

Kerjakan mandiri selama 8 menit, lalu bahas bersama. Kunci tersedia pada versi handout.

Kegiatan Mahasiswa di Kelas: P4-A

Tabel aturan semantik dan kode antara

1. Lengkapi tabel aturan ekspresi dan pernyataan.
2. Pilih program dengan deklarasi, assignment, dan percabangan/perulangan; gambar pohon sintaksnya.
3. Terapkan setiap aturan pada simpul dan catat atribut `type` serta `code`.
4. Tuliskan postfix dan three-address code untuk setiap ekspresi.
5. Tuliskan quadruple dan three-address code untuk setiap pernyataan.
6. Uji minimal 15 ekspresi dan 5 pernyataan.
7. Kumpulkan tabel, pohon teranotasi, dan keluaran kode antara.

Tugas dan Artefak

Artefak Sub-CPMK 1.5

Tabel aturan semantik berbasis SDD untuk ekspresi dan pernyataan, pohon sintaks teranotasi, dan keluaran pengujian aturan.

- Tabel ekspresi dan pernyataan lengkap.
- Pohon sintaks teranotasi dengan tipe dan atribut kode.
- Minimal 15 ekspresi dan 5 pernyataan diuji.
- Postfix, quadruple, dan three-address code tersedia.
- Laporan dan keluaran pengujian diserahkan.

P4-A menjadi bagian Praktikum 4 (12%) yang dilaporkan pada Pertemuan 14.

Rangkuman dan Jembatan

- Analisis semantik memeriksa makna: deklarasi, tipe, fungsi, kontrol alur, dan inisialisasi.
- Attribute grammar memakai atribut synthesized dan inherited.
- SDD mendeskripsikan aturan; translation scheme menentukan urutan eksekusi.
- Tabel aturan ekspresi dan pernyataan diuji pada pohon sintaks konkret.
- `TYPE_ERROR` meredam pesan turunan dan menjaga diagnostik tetap fokus.

Pertemuan berikutnya

Pertemuan 14 mendalami symbol table bersarang, manajemen scope, dan type checking dalam C, lalu menutup Praktikum 4.